

深度学习自然语言处理作业报告

孟子焱
2638492711@qq.com

摘要

本报告基于标准Transformer架构，通过消融实验研究两个核心模块——残差连接和独立的 Q、K、V 投影矩阵的必要性。我首先完成了实验环境的配置，在之前做实验配置的yolov8环境基础上加入torchtext 的安装以及 Multi30k 数据集的离线获取。为降低计算开销，训练仅使用数据集中前 2000 条句子对。本实验在 Multi30k 英德翻译数据集上训练了基线模型、无残差模型以及 K、V 共享（即只保留 Q 与 K，合并 V 与 K）模型。实验记录了训练/验证损失及 BLEU 分数。结果表明：残差连接移除后，验证损失增加 35.5%，BLEU 从 19.4 降至 10.5；K、V 共享后，验证损失略有增加（0.7%），但 BLEU 从 19.4 降至 15.4（下降 20.6%）。本报告验证了残差连接和独立 Value 投影对 Transformer 性能的关键作用。

引言

Transformer自2017年提出以来，已成为自然语言处理领域的基础架构。它完全基于注意力机制，摒弃了循环神经网络和卷积神经网络的固有局限。其中两个关键设计——残差连接和多头注意力的独立Q、K、V投影——被认为对模型性能至关重要。残差连接保证了梯度能有效传播，使得深层网络可训练；独立的 Q、K、V 投影则允许模型在不同表征子空间中捕获依赖关系。

本报告通过控制变量法，分别移除残差连接和强制 K、V 共享（即令 $V = K$ ），定量评估这两个模块对翻译任务的影响。

模型介绍

标准 Transformer 架构

Transformer 采用编码器-解码器结构。编码器由 N 个相同层堆叠（原论文 $N=6$ ），每层包含两个子层：

多头自注意力：允许每个位置关注序列中的所有位置。

逐位置前馈网络：对每个位置的向量进行非线性变换。

每个子层后均跟随残差连接和层归一化。解码器与编码器结构类似，但在两个子层之间额外插入一个编码器-解码器注意力子层，用于关注源序列。此外，解码器的自注意力使用因果掩码，防止当前位置关注未来位置。

为了感知序列顺序，Transformer 使用了正弦余弦位置编码，直接与词嵌入相加后输入网络。

本实验的实现

为适应CPU环境和加速实验迭代，本实验实现了一个完整但轻量化的

Transformer:

保留所有核心组件：交叉注意力、因果掩码、位置编码、残差连接、多头注意力。

简化超参数：嵌入维度128，注意力头数2，编码器/解码器层数 2，前馈网络中间维度 512。

训练数据：Multi30k 数据集，仅使用前 2000 个句子对。

评估指标：交叉熵损失（训练/验证）和 BLEU 分数（测试集）。

模型通过两个布尔参数控制消融：

res：是否保留残差连接。

share_kv：是否将 K、V 投影矩阵合并（即令 $V = K$ ）。

实验环境与配置过程

软硬件环境

操作系统：Windows 11

Python 版本：3.8

深度学习框架：PyTorch 1.10.1

硬件：CPU

环境配置详细步骤

实验环境配置经历了以下关键步骤：

使用之前配置过的yolov8环境。

安装 torchtext：由于官方源 SSL 证书问题，切换到阿里云镜像源：`pip install torchtext==0.11.2-https://mirrors.aliyun.com/pypi/simple/--trusted-host mirrors.aliyun.com`。

安装辅助库：`pip install sacrebleu tqdm`。

手动下载 Multi30k 数据集：由于原始链接 SSL 证书过期，手动下载 `training.tar.gz`、`validation.tar.gz`、`mmt16_task1_test.tar.gz` 并解压至 `.data/multi30k/`，得到 `train.en`, `train.de`, `val.en`, `val.de`, `test.en`, `test.de`。

为加速 CPU 训练，仅使用训练集中的前 2000 个句子对（代码中通过 `[:2000]` 切片实现）。验证集和测试集保持完整（约 1000 句对）。

超参数一览

词嵌入维度： 128

注意力头数： 2

编码器/解码器层数： 2

前馈网络中间维度 : 512

优化器： Adam（学习率 $1e-4$ ）

损失函数： 交叉熵

训练轮数： 10

批次大小： 16

消融实验设计

设计了三个模型变体，所有变体共享上述超参数和数据划分。

基线： 完整 Transformer，作为性能上界。

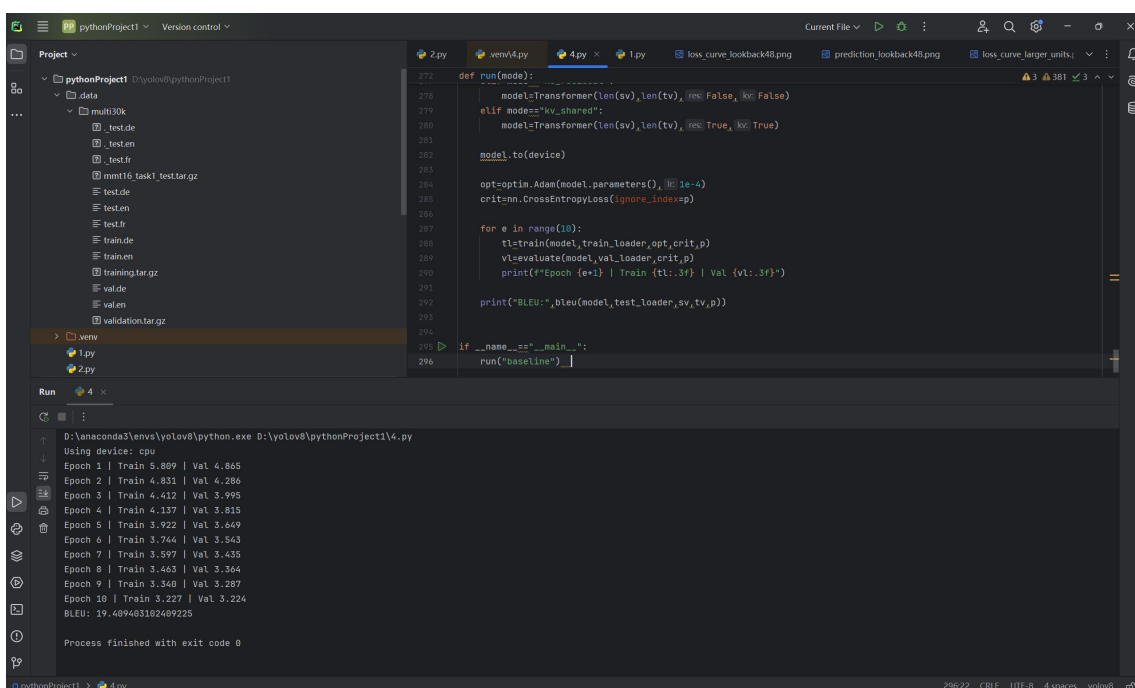
无残差： 移除所有残差连接，验证残差连接对梯度传播和最终性能的影响。

K、V共享：强制V=K（只保留Q与K），验证独立的Value 投影矩阵是否必要。
每个模型训练10个epoch，每个epoch后计算训练损失和验证损失，训练结束后在测试集上计算 BLEU 分数

实验结果

表 1: 实验结果展示

模型	Epoch 1 (Train/Val)	Epoch 5 (Train/Val)	Epoch 10 (Train/Val)	最终验证损失	测试 BLEU
基线	5.809 / 4.865	3.922 / 3.649	3.227 / 3.224	3.224	19.41
无残差	5.870 / 5.060	4.791 / 4.465	4.503 / 4.369	4.369	10.46
K、V 共享	5.813 / 4.866	3.963 / 3.676	3.277 / 3.247	3.247	15.42



```
def run(mode):
    model=Transformer(len(sv),len(tv), res=False, kv=False)
    elif mode=="kv_shared":
        model=Transformer(len(sv),len(tv), res=True, kv=True)
    model.to(device)
    opt=optim.Adam(model.parameters(), 1e-4)
    crit=nn.CrossEntropyLoss(ignore_index=p)
    for e in range(10):
        tl=train(model, train_loader, opt, crit, p)
        vl=evaluate(model, val_loader, crit, p)
        print(f"Epoch {e+1} | Train {tl:.3f} | Val {vl:.3f}")
    print("BLEU:", bleu(model, test_loader, sv, tv, p))
if __name__=="__main__":
    run("baseline")
```

Run

D:\anaconda3\envs\yolov8\python.exe D:\yolov8\pythonProject1\4.py

Using device: cpu

Epoch 1 | Train 5.809 | Val 4.865

Epoch 2 | Train 4.831 | Val 4.286

Epoch 3 | Train 4.412 | Val 3.995

Epoch 4 | Train 4.137 | Val 3.815

Epoch 5 | Train 3.922 | Val 3.649

Epoch 6 | Train 3.744 | Val 3.543

Epoch 7 | Train 3.597 | Val 3.435

Epoch 8 | Train 3.463 | Val 3.364

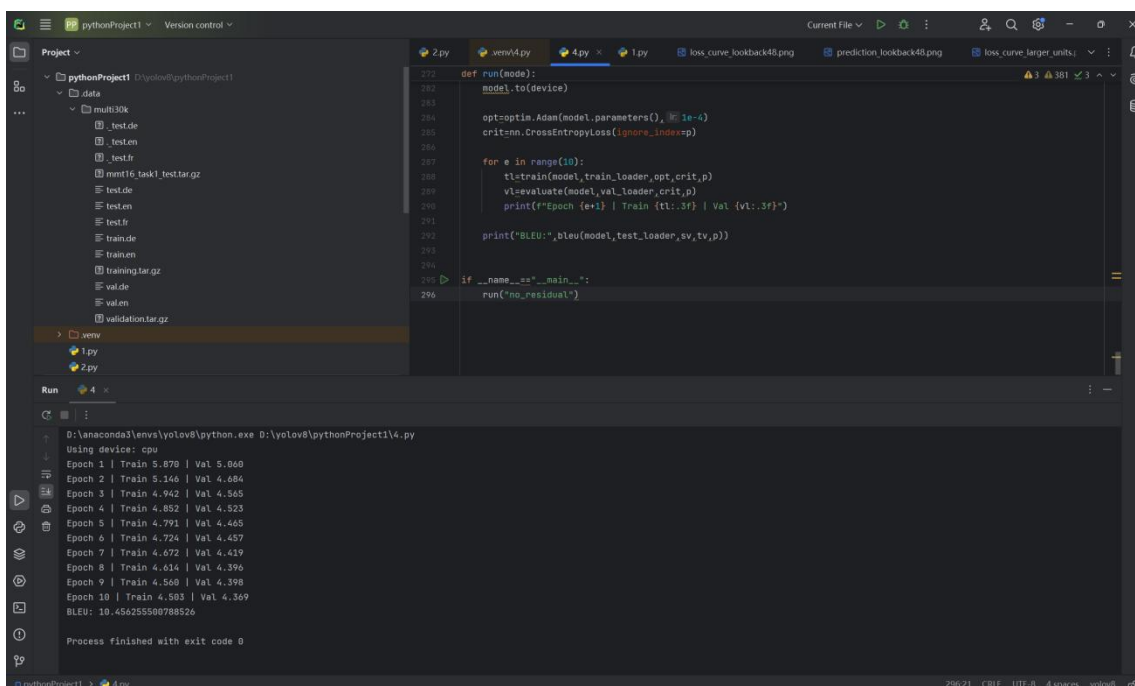
Epoch 9 | Train 3.340 | Val 3.287

Epoch 10 | Train 3.227 | Val 3.224

BLEU: 19.409403102409225

Process finished with exit code 0

图 1: 基线模型运行结果展示



```
def run(mode):
    model.to(device)
    opt=optim.Adam(model.parameters(), 1e-4)
    crit=nn.CrossEntropyLoss(ignore_index=p)
    for e in range(10):
        tl=train(model, train_loader, opt, crit, p)
        vl=evaluate(model, val_loader, crit, p)
        print(f"Epoch {e+1} | Train {tl:.3f} | Val {vl:.3f}")
    print("BLEU:", bleu(model, test_loader, sv, tv, p))
if __name__=="__main__":
    run("no_residual")
```

Run

D:\anaconda3\envs\yolov8\python.exe D:\yolov8\pythonProject1\4.py

Using device: cpu

Epoch 1 | Train 5.870 | Val 5.060

Epoch 2 | Train 5.146 | Val 4.684

Epoch 3 | Train 4.942 | Val 4.565

Epoch 4 | Train 4.852 | Val 4.523

Epoch 5 | Train 4.791 | Val 4.465

Epoch 6 | Train 4.724 | Val 4.457

Epoch 7 | Train 4.672 | Val 4.419

Epoch 8 | Train 4.614 | Val 4.396

Epoch 9 | Train 4.568 | Val 4.398

Epoch 10 | Train 4.503 | Val 4.369

BLEU: 10.456255500788526

Process finished with exit code 0

图 2：无残差模型运行结果展示

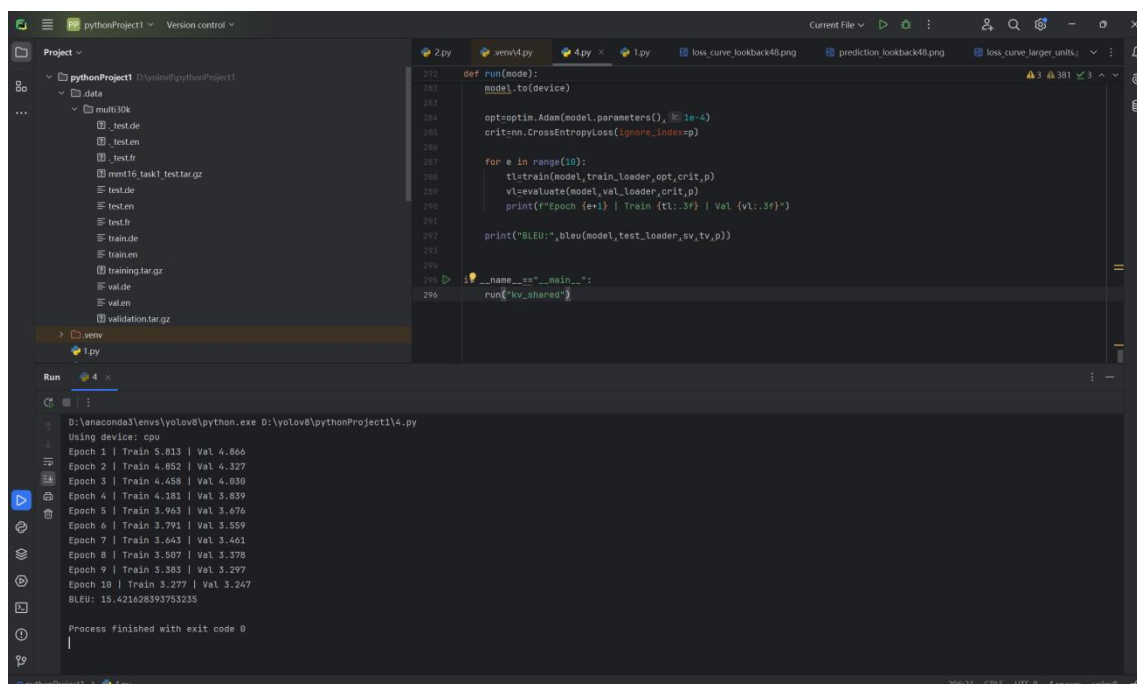


图 3：K、V 共享模型运行结果展示

结果分析：

基线模型：训练损失从5.809平稳下降至3.227，验证损失从4.865降至3.224，收敛良好。测试BLEU为19.41。考虑到仅使用2000句对、2层网络和CPU训练，这是一个合理的结果，表明完整 Transformer 能够有效学习翻译模式。

无残差模型：最终验证损失为4.369，比基线高出1.145（相对恶化约35.5%）。BLEU从19.41骤降至10.46（下降约46%）。从训练曲线看，无残差模型在第5轮后损失下降近乎停滞，而基线持续下降。这证实了残差连接为梯度提供了“高速公路”，是深层网络可训练的必要条件。即使在仅2层的网络中，移除残差也会导致明显性能损失；对于标准6层 Transformer，影响会更大。

K、V 共享模型：最终验证损失为 3.247，仅比基线高出 0.023（相对恶化约 0.7%），差异很小。但 BLEU 从 19.41 降至 15.42（下降约 20.6%），表明独立的 Value 投影对生成质量有显著影响。分析：当V=K时，注意力输出的表达范围受限，模型难以从不同表征子空间中提取丰富信息。尽管交叉熵损失差异不大，但BLEU对这种细微变化更为敏感。这验证了多头注意力中“每个头独立投影”设计的必要性。

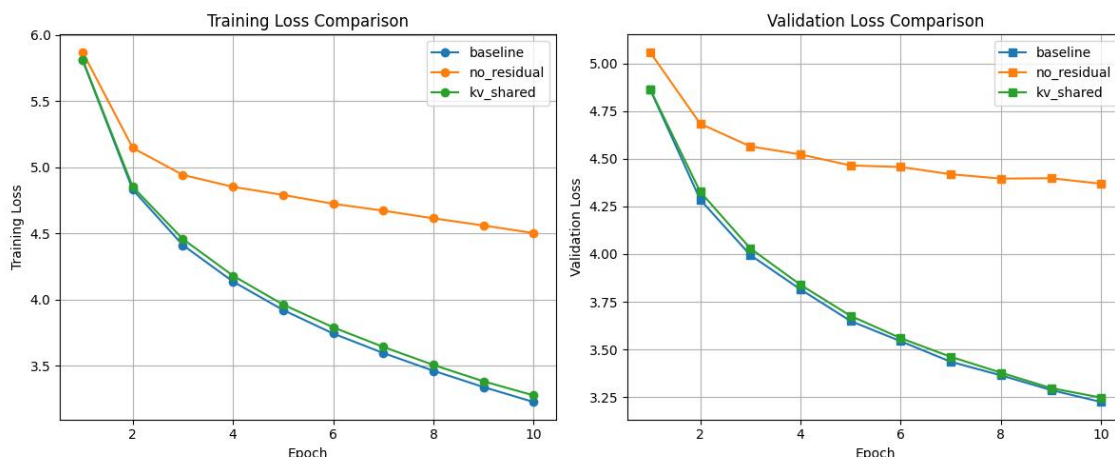


图 4：三个模型的训练损失及验证损失对比

由此可得：

残差连接是Transformer训练稳定性的基石。无残差模型几乎无法优化，验证损失高且 BLEU 极低。

独立的Value投影对翻译质量贡献显著，其重要性在BLEU上体现得比损失函数更明显。

结论

本报告基于完整实现的 Transformer 模型，通过消融实验得出以下结论：

残差连接是Transformer能够有效训练的必要组件。移除残差连接后，模型收敛缓慢，验证损失大幅升高（+35.5%），BLEU 下降 46%。

独立的Value投影矩阵对模型生成质量有显著贡献。K、V 共享后，BLEU 下降20.6%，说明独立的V投影允许注意力输出更丰富的表征，对翻译任务尤为重要。

环境配置的挑战：实际实验中遇到了 SSL 证书问题、手动下载数据集、CPU 训练缓慢等困难，通过切换镜像源、手动整理数据、缩减训练集等方法成功解决。这些经验对后续深度学习实验具有参考价值。

未来工作可进一步探索不同位置编码（作业2.1）、CNN替代架构（2.4）以及动态稀疏注意力等创新方向（2.5）。

参考文献

[1] Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems, pp. 5998–6008.